Autonomous AI Trading & Risk Management System

Mentor Discussion / Project Proposal

1. Project Overview

The proposed project is an intelligent trading platform that allows a user to define their own trading strategy and then evaluates that strategy using market data, machine-learning techniques, and automated risk-management rules.

The key idea is that the AI does not simply predict whether the market will go up or down. Instead, the user's strategy generates the initial trading signal, while an AI-based market-regime and risk layer evaluates whether the signal should be accepted, rejected, or traded with modified risk.

The system can support backtesting, paper trading, performance analysis, and—subject to API and safety constraints—automated execution.

2. Problem Statement

Many trading systems either rely entirely on fixed technical rules or attempt to predict prices directly using machine learning. A fixed strategy may perform well in one market condition and poorly in another, while a pure price-prediction model can be difficult to interpret and may suffer from overfitting.

This project proposes a hybrid approach: keep the trader's strategy as the primary signal generator and use AI/ML as an additional decision-support and risk-control layer. The system will test whether this additional intelligence improves risk-adjusted performance compared with the original strategy alone.

3. Main Objectives

- Allow users to enter configurable trading rules.
- Generate entry and exit signals from those rules.
- Identify market regimes such as bullish, bearish, sideways, low-volatility, and high-volatility conditions.
- Use AI/ML to score or filter strategy signals.
- Calculate position size from account equity, stop-loss distance, and permitted risk.
- Dynamically adapt stop-loss/take-profit parameters using volatility measures where appropriate.
- Monitor portfolio exposure and trading risk.
- Backtest the strategy on historical data.
- Support paper trading using live or simulated market data.
- Produce performance metrics such as P&L;, win rate, maximum drawdown, Sharpe ratio, profit factor, and risk/reward statistics.
- Compare the original strategy against AI-filtered and dynamically risk-managed versions.

4. How the System Works

The proposed workflow is:

1. User Strategy Input → The user defines entry, exit, stop-loss, take-profit, timeframe, indicators, trading session, and risk limits.

2. Market Data → Historical or live OHLCV data and selected technical/market features are supplied to the system.

3. Strategy Signal Engine → The user's rules generate BUY, SELL, or NO-TRADE signals.

4. Market Regime Detection → An ML model classifies the current market environment, for example trend, range, or high-volatility conditions.

5. AI Signal Filter → The system evaluates whether the strategy signal is suitable for the current regime and assigns a confidence/quality score.

6. Risk Engine → Position size, stop-loss distance, portfolio exposure, and daily risk limits are calculated.

7. Decision Layer → The system accepts, rejects, or modifies the proposed trade according to predefined risk rules.

8. Execution Layer → The trade is sent to a paper-trading simulator or, in an advanced version, to a supported broker/exchange API.

9. Monitoring & Analytics → The system records every trade and continuously calculates performance and risk metrics.

5. User-Defined Strategy Module

The platform should allow the user to define rules without changing the core program. Example parameters include:

- Entry condition: EMA crossover, RSI threshold, breakout, price action condition, etc.
- Exit condition: opposite signal, target reached, time-based exit, etc.
- Stop Loss: fixed points, percentage, ATR-based distance, or another rule.
- Take Profit: fixed risk/reward ratio, percentage, ATR multiple, or rule-based target.
- Timeframe: 1-minute, 5-minute, 15-minute, 1-hour, daily, etc.
- Trading session: selected market hours.
- Risk per trade: for example, a user-defined percentage of account equity.
- Maximum number of simultaneous positions.
- Maximum daily loss and other safety limits.

6. AI / Machine Learning Component

The AI component is intended to complement rather than replace the trading strategy.

Possible ML tasks include:

- Market-regime classification: trend, range, high volatility, low volatility.
- Signal-quality classification: estimate whether a strategy signal has historically been favorable under similar conditions.
- Trade filtering: reject signals with low predicted quality.
- Volatility estimation: help select appropriate stop-loss/take-profit distances.
- Anomaly detection: identify unusual market conditions where trading should be restricted.

Candidate models can include Random Forest, XGBoost, Logistic Regression, or other interpretable classifiers. A more advanced version could investigate sequence models, but the initial project should prioritize a robust, explainable model over unnecessary complexity.

7. Risk Management Engine

Risk management is a central part of the project.

For example, position size can be calculated approximately as:

Position Size = Maximum Money Risk / Risk Per Unit

where:

Maximum Money Risk = Account Equity × Risk Percentage

Risk Per Unit depends on the entry price and stop-loss distance, adjusted for the instrument's contract/share/point value.

The engine can also enforce:

- Maximum risk per trade
- Maximum total open risk
- Maximum daily loss
- Maximum number of simultaneous positions
- Maximum portfolio exposure
- Trading suspension after predefined loss limits
- Volatility-based position-size reduction

This makes the project more than a signal-generation system; it becomes a complete decision and risk-control framework.

8. Backtesting and Experimental Design

The project should compare three configurations:

A. Baseline Strategy

The user's original strategy trades every valid signal without an AI filter.

B. Strategy + AI Filter

The same strategy generates signals, but the AI decides whether a signal is sufficiently favorable for the current market regime.

C. Strategy + AI + Dynamic Risk Management

The AI-filtered strategy is combined with position sizing, volatility-aware stops/targets, exposure controls, and portfolio-level risk limits.

All three versions should be evaluated on the same historical periods and under consistent assumptions. The data should be divided into training, validation, and out-of-sample test periods to reduce overfitting and provide a fair comparison.

9. Performance Metrics

The system should report:

- Total return / net P&L;
- Number of trades
- Win rate

- Average win and average loss
- Profit factor
- Maximum drawdown
- Sharpe ratio
- Sortino ratio, if implemented
- Average risk/reward
- Expectancy per trade
- Long vs. short performance
- Performance by market regime
- Daily/weekly/monthly equity curve

The most important goal is not simply maximizing return. The project should investigate whether AI filtering and risk management improve risk-adjusted returns and reduce undesirable drawdowns.

10. Proposed Technology Stack

Possible implementation stack:

- Programming: Python
- Data processing: Pandas, NumPy
- Machine Learning: Scikit-learn and/or XGBoost
- Technical indicators: pandas-ta or a custom indicator module
- Backtesting: Backtrader, vectorbt, or a custom backtesting engine
- Visualization: Plotly and/or Matplotlib
- Database: SQLite initially; PostgreSQL for an expanded version
- API/backend: FastAPI
- Frontend/dashboard: Streamlit or a web frontend
- Deployment: Local system initially, followed by cloud deployment if required
- Market data/API: A suitable historical/live data provider and, for the advanced stage, a supported broker/exchange API.

11. System Architecture

The architecture can be divided into the following modules:

User Interface

↓

Strategy Configuration

↓

Market Data Layer

↓

Strategy Signal Engine

↓

AI Market-Regime & Signal Filter

↓

Risk Management Engine

↓

Decision / Execution Engine

↓

Trade Database

↓

Performance Dashboard

This modular design allows each component to be tested independently and makes it easier to replace the AI model or strategy without redesigning the complete platform.

12. Expected Outcome

The final system should demonstrate whether a conventional rule-based strategy can be improved by adding an AI-driven market-condition filter and systematic risk management.

The expected deliverable is a working prototype that can accept a configurable strategy, run historical backtests, classify market conditions, filter signals, calculate risk-adjusted position sizes, simulate trades, and display performance results.

The project should produce a quantitative comparison showing whether the AI and risk-management layers improve metrics such as drawdown, Sharpe ratio, expectancy, and consistency.

13. Innovation / Why This Project Is Different

The main novelty is the separation of responsibilities:

- The trader defines the strategy.
- The strategy creates the initial signal.
- AI evaluates the market context and signal quality.
- The risk engine determines how much capital can be exposed.
- The execution layer handles the trade.
- The analytics layer measures whether the additional intelligence actually helped.

Therefore, the project is not simply another stock-price prediction model. It combines trading strategy design, machine learning, quantitative evaluation, risk management, and automation in one measurable framework.

14. Scope and Safety

For the academic project, the recommended progression is:

Phase 1 — Strategy configuration and historical backtesting.

Phase 2 — ML market-regime classification and signal filtering.

Phase 3 — Position sizing and portfolio risk management.

Phase 4 — Paper trading with live/simulated market data.

Phase 5 — Optional API execution as an advanced extension.

Real-money automated trading should not be necessary to demonstrate the academic value of the project. Paper trading provides a safer and more controllable environment for evaluation.

15. Suggested Final Project Title

Autonomous AI Trading & Risk Management System: A Hybrid Strategy, Market-Regime Filtering and Automated Risk-Control Platform

One-Minute Mentor Pitch

Our project is an Autonomous AI Trading and Risk Management System. Instead of asking AI to blindly predict BUY or SELL, we let the user define a normal trading strategy first. That strategy generates the initial signal, and an AI layer studies the market regime—such as trending, sideways, or highly volatile conditions—to decide whether the signal is worth taking. A separate risk engine calculates position size, stop loss, take profit, and portfolio exposure. We can then compare the original strategy, the strategy with AI filtering, and the strategy with AI plus dynamic risk management using historical backtesting and paper trading. This gives us a measurable way to determine whether AI actually improves trading performance and risk, rather than simply claiming that an ML model can predict prices.